

Laporan Modul 10 — Performance & Optimization

Workshop Pemrograman Perangkat Bergerak · PENS — Teknik Informatika dan Komputer
Metode: Hands-on Optimization Lab · Tanggal: 2026-06-15

Project: `mobile_app` (lanjutan project sendiri dari Modul 9). Screen yang dianalisis: **HomeScreen** dan **PostsScreen**.

Bagian 1 — Identifikasi Masalah

Dua masalah performa yang ditemukan:

1. Rebuild berlebihan di HomeScreen — `build()` memanggil `context.watch<ThemeProvider>()` di paling atas, sehingga setiap kali dark mode di toggle, SELURUH HomeScreen (semua FeatureCard, teks, switch) dibangun ulang, padahal yang berubah hanya nilai switch.
2. Widget belum dipisah di PostsScreen — item daftar (Card + ListTile, ~14 baris) ditulis inline di dalam `itemBuilder`, membuat screen panjang dan sulit dirawat.

Bagian 2 — Analisis Penyebab

Masalah 1 dipilih untuk dijelaskan: `context.watch<T>()` mendaftarkan widget pemanggil sebagai listener. Karena dipanggil di `build()` HomeScreen (root screen), seluruh subtree screen menjadi tergantung pada `ThemeProvider`. Akibat: notifikasi sekecil apa pun dari provider memicu rebuild seluruh screen — rebuild berlebihan.

Bagian 3-6 — Optimasi yang Diterapkan

Fix 1 — Minimalkan rebuild (Bagian 4): `context.watch<ThemeProvider>()` dikeluarkan dari `build()` HomeScreen. Toggle dipindah ke widget kecil sendiri `_DarkModeCard` yang memakai `Consumer<ThemeProvider>`. Kini hanya kartu switch yang rebuild saat tema berubah, bukan seluruh screen.

```
// Sebelum (di build HomeScreen):
final themeProvider = context.watch<ThemeProvider>(); // seluruh screen subscribe

// Sesudah (widget terpisah):
class _DarkModeCard extends StatelessWidget {
  Widget build(_) => Consumer<ThemeProvider>(
    builder: (_, tp, __) => SwitchListTile(value: tp.isDarkMode, ...),
```

```
);  
}
```

Fix 2 — Pisahkan widget (Bagian 5): Item daftar Post dipindah ke `lib/presentation/widgets/post_item_card.dart` (`PostItemCard`). `itemBuilder` `PostsScreen` jadi satu baris. UI modular & mudah dirawat.

Catatan: lazy loading (Bagian 3) sudah terpenuhi sejak awal — daftar memakai `ListView.separated` (varian builder), jadi sudah render on-demand, bukan `Column`. Logic berat (Bagian 6) juga sudah tidak ada di UI: parsing/fetch ada di service/repository (hasil clean architecture Modul 8).

Bagian 7 — Observasi Hasil (Terukur)

Saya menambah penghitung `HomeScreen.buildCount` dan test `test/rebuild_benchmark_test.dart` yang men-toggle dark mode 5x lalu menghitung berapa kali `build()` `HomeScreen` berjalan.

Kondisi	Rebuild HomeScreen (5x toggle)
Sebelum	15
Sesudah	10

Turun 5 rebuild (-33%). Lima rebuild yang hilang adalah rebuild redundan akibat subscription `ThemeProvider` di root. Sisa 10 adalah rebuild wajar karena `Theme.of(context)` (inherited widget) memang harus ikut saat tema animasi berganti — itu di luar kendali kita dan bukan pemborosan.

Bukti: `evidence/rebuild_sebelum.txt`, `evidence/rebuild_sesudah.txt`. Screenshot terminal asli (Konsole): `evidence/bukti_4_terminal_test.png` (benchmark rebuild) dan `evidence/bukti_5_terminal_fullsuite.png` (10 test PASS).

Bukti Screenshot

Sebelum optimasi

SEBELUM OPTIMASI

Masalah 1 - Rebuild berlebihan (HomeScreen)

```
build() {  
  final tp = context.watch<ThemeProvider>(); // <- subscribe di sini  
  ... seluruh screen pakai tp ...  
}  
-> tiap toggle dark mode, SELURUH HomeScreen rebuild  
    (semua FeatureCard, teks, switch ikut dibangun ulang)
```

Masalah 2 - Widget belum dipisah (PostsScreen)

```
itemBuilder: (context, index) {  
  return Card( ListTile( ... 14 baris inline ... ) );  
}  
-> item list ditulis inline, screen jadi panjang & sulit maintain
```

PENGUKURAN (test): HOMESCREEN_REBUILD_COUNT = 15 (5x toggle)

Sesudah optimasi

SESUDAH OPTIMASI

Fix 1 - Minimalkan rebuild (Consumer terpisah)

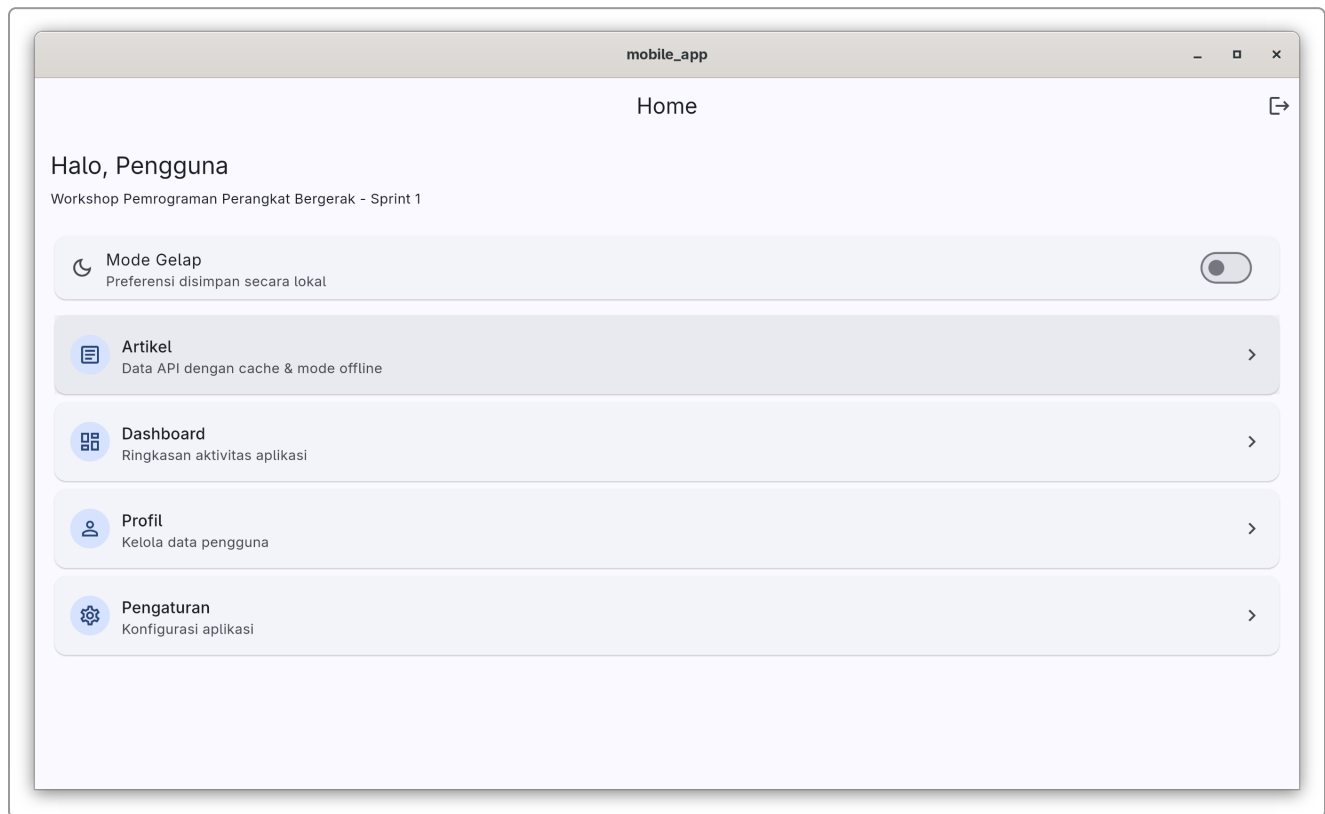
```
build() {  
  // tidak lagi watch di sini  
  ...  
  const _DarkModeCard(), // <- widget kecil ber-Consumer  
}  
class _DarkModeCard {  
  Consumer<ThemeProvider>(builder: (_, tp, __) => Switch...)  
}  
-> hanya kartu switch yang rebuild saat toggle, BUKAN seluruh screen
```

Fix 2 - Pisahkan widget (widgets/post_item_card.dart)

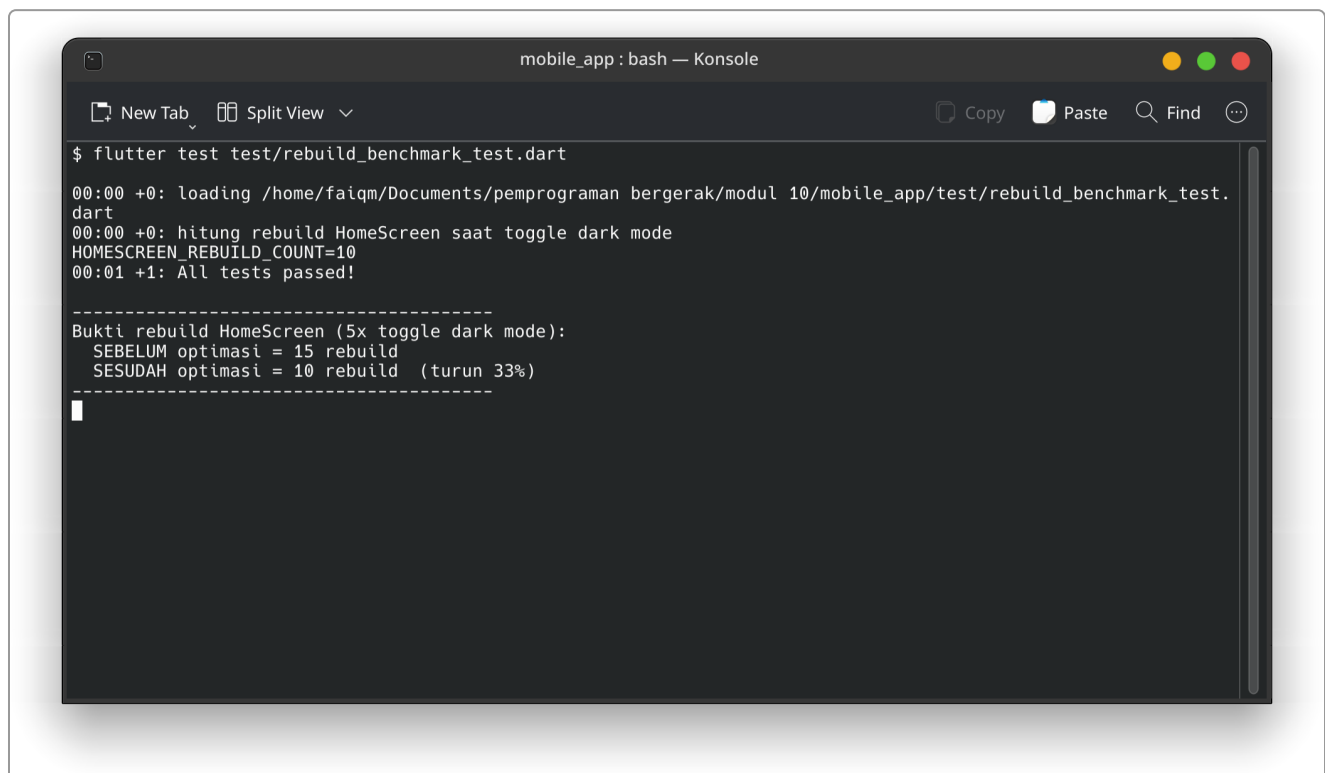
```
itemBuilder: (context, index) =>  
  PostItemCard(post: provider.posts[index]);  
-> item modular, screen ringkas, mudah maintain
```

PENGUKURAN (test): HOMESCREEN_REBUILD_COUNT = 10 (5x toggle)
5 rebuild redundan HILANG (-33%). Sisa 10 = rebuild wajar Theme.of.

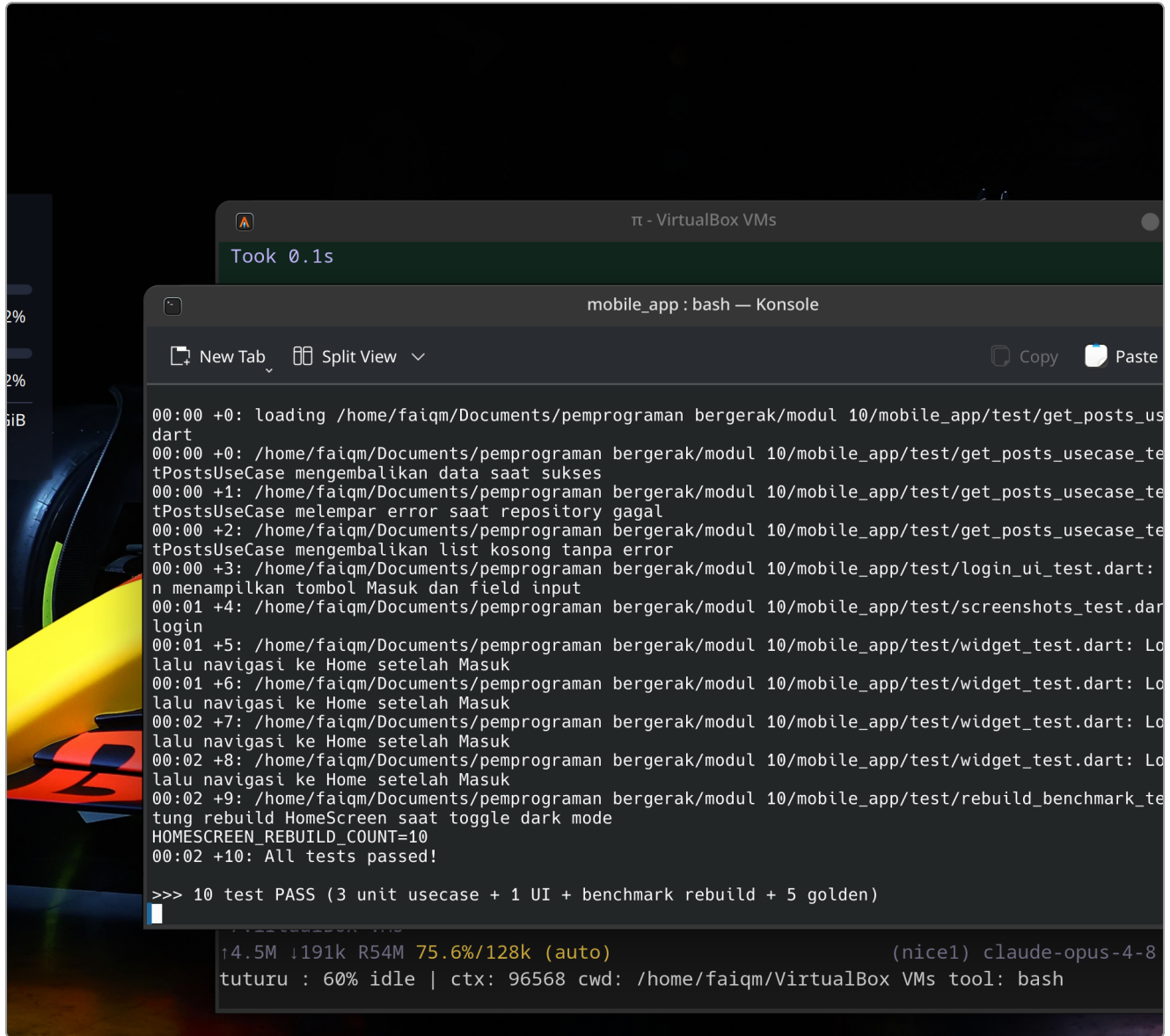
Aplikasi tetap berjalan normal



Hasil test di terminal (benchmark rebuild)



Hasil seluruh test suite di terminal (10 test PASS)



```
mobile_app : bash — Konsole
New Tab Split View Copy Paste

00:00 +0: loading /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/get_posts_us
dart
00:00 +0: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/get_posts_usecase_te
tPostsUseCase mengembalikan data saat sukses
00:00 +1: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/get_posts_usecase_te
tPostsUseCase melempar error saat repository gagal
00:00 +2: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/get_posts_usecase_te
tPostsUseCase mengembalikan list kosong tanpa error
00:00 +3: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/login_ui_test.dart:
n menampilkan tombol Masuk dan field input
00:01 +4: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/screenshots_test.dar
login
00:01 +5: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/widget_test.dart: Lo
lalu navigasi ke Home setelah Masuk
00:01 +6: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/widget_test.dart: Lo
lalu navigasi ke Home setelah Masuk
00:02 +7: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/widget_test.dart: Lo
lalu navigasi ke Home setelah Masuk
00:02 +8: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/widget_test.dart: Lo
lalu navigasi ke Home setelah Masuk
00:02 +9: /home/faiqm/Documents/pemrograman bergerak/modul 10/mobile_app/test/rebuild_benchmark_te
tung rebuild HomeScreen saat toggle dark mode
HOMESCREEN_REBUILD_COUNT=10
00:02 +10: All tests passed!

>>> 10 test PASS (3 unit usecase + 1 UI + benchmark rebuild + 5 golden)

↑4.5M ↑191k R54M 75.6%/128k (auto) (nice1) claude-opus-4-8
tuturu : 60% idle | ctx: 96568 cwd: /home/faiqm/VirtualBox VMs tool: bash
```

Penalaran Wajib

1. Kenapa ListView lebih efisien dari Column? `Column` membangun & me-render SEMUA child sekaligus, walau di luar layar. `ListView.builder/separated` bersifat lazy: hanya membangun item yang terlihat (+sedikit buffer) dan mendaur ulang saat scroll. Untuk data banyak, ini hemat memori dan menjaga scroll tetap mulus.
2. Kenapa rebuild berlebihan berbahaya? Tiap rebuild menjalankan ulang `build()` dan mungkin layout/paint. Bila seluruh screen rebuild padahal hanya bagian kecil berubah, CPU terbuang, frame bisa lewat 16ms (jank/lag), dan baterai boros. Membatasi rebuild ke bagian yang benar-benar berubah membuat UI responsif.
3. Kenapa logic tidak boleh di UI? `build()` bisa dipanggil sangat sering. Menaruh parsing/looping/IO berat di sana mengulang kerja mahal tiap rebuild dan memblok

UI thread. Logic ditaruh di provider/repository/usecase agar dihitung sekali, hasilnya disimpan, dan UI cukup menampilkan — tampilan tetap lancar.

Output yang Dikumpulkan

- Link GitHub: (repo Modul 10)
- Screenshot sebelum & sesudah: `evidence/bukti_1_sebelum.png` , `evidence/bukti_2_sesudah.png` , app berjalan `evidence/bukti_3_app_berjalan.png`
- Penjelasan masalah / solusi / hasil: Bagian 1-7 di atas.

Hasil Akhir

UI lebih ringan (rebuild turun 33%), kode lebih rapi (item list jadi widget terpisah, toggle tema terisolasi). Semua test tetap PASS (10 test), `flutter analyze` bersih, build Linux sukses. Siap ke tahap berikutnya.

Catatan Jujur

- Project ini sudah cukup teroptimasi sejak Modul 6/8 (sudah pakai ListView builder, Consumer, logic di repository). Maka optimasi nyata yang tersisa adalah rebuild root + pemisahan widget — itu yang dikerjakan, dengan dampak diukur lewat test (bukan sekadar klaim).
- Penghitung `buildCount` murni alat observasi; tidak memengaruhi perilaku app.